

Week 1 outputs

Design Patterns and Principles:

Exercise 1: Implementing the Singleton Pattern

```
Logger instance created.  
[LOG] Application started  
[LOG] User logged in  
  
Same instance? true  
Logger1 hashCode: 681842940  
Logger2 hashCode: 681842940
```

Explanation :

The Singleton Pattern ensures that only one instance of a class is created throughout the application. In this exercise, the Logger class is implemented as a Singleton — the message "Logger instance created." appears only once even though getInstance() is called twice, proving only one object is created. Both logger1 and logger2 refer to the same instance, which is confirmed by the output "Same instance? true" and identical hashCode values showing they point to the same memory location. This pattern is useful in real-world applications like logging systems where a single shared instance is needed across the entire application.

Exercise 2: Implementing the Factory Method Pattern

```
=== Document Management System ===  
  
Opening Word Document (.docx)  
Opening PDF Document (.pdf)  
Opening Excel Document (.xlsx)  
  
All documents created using Factory Method!
```

Explanation :

The Factory Method Pattern is a creational design pattern that creates objects without directly specifying the exact class, letting the factory decide which object to instantiate. In this exercise, I built a Document Management System where WordFactory, PdfFactory and ExcelFactory each create their respective document objects — WordDocument, PdfDocument and ExcelDocument — all implementing a common Document interface with an open() method. The main class simply calls createDocument() on each factory and the correct document is created and opened automatically. The output confirming all three document types are created successfully proves that the Factory Method Pattern is working correctly.

Data structures and Algorithms

Exercise 2: E-commerce Platform Search Function

```
=== E-commerce Product Search ===
Available: Camera, Laptop, Phone, Tablet, Watch

Enter product to search: Laptop

Linear Search : Found! ID=2, Category=Electronics
Binary Search : Found! ID=2, Category=Electronics

Linear Search -> O(n)      - checks one by one
Binary Search -> O(log n) - splits in half each time
```

Explanation :

In this exercise, I implemented Linear Search and Binary Search for a product search system using a Product class with productId, productName and category. Linear Search checks each product one by one with $O(n)$ complexity, while Binary Search splits the sorted list in half each time making it faster with $O(\log n)$ complexity. Both algorithms display the product ID and category when found, and Binary Search is more suitable for large platforms where speed and performance matter.

Exercise 7: Financial Forecasting

```
=== Financial Forecasting Tool ===

Enter initial amount (Rs.)      : 10000
Enter annual growth rate (%)    : 52
Enter number of years           : 5

--- Forecast Results ---
Year 1 : Rs.15200.00
Year 2 : Rs.23104.00
Year 3 : Rs.35118.08
Year 4 : Rs.53379.48
Year 5 : Rs.81136.81

Final Value after 5 years : Rs.81136.81

Time Complexity : O(n) - recurses once per year
```

Explanation :

This program implements a **financial forecasting tool** using recursion to calculate the future value of an investment. The recursive method increases the amount year by year based on the given growth rate until the required number of years is reached. The program takes the initial amount, annual growth rate, and duration as input and displays the forecast for each year. The time complexity of the recursive calculation is $O(n)$ because the function is called once for each year.

TDD using JUnit5 and Mockito

Exercise 1: Setting Up Junit

```
/*<!-- JUnit Dependency in pom.xml -->
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>*/
```

```
=== JUnit Setup - Calculator Test ===
```

```
Enter first number : 50
Enter second number : 52
```

```
--- Test Results ---
```

```
Add      : 50 + 52 = 102
Subtract  : 50 - 52 = -2
Multiply  : 50 x 52 = 2600
Divide    : 50 / 52 = 0
```

```
All tests passed successfully!
```

Explanation:

JUnit is a testing framework used in Java to verify that methods return correct results. In this exercise, I created a Calculator class with four basic operations — add, subtract, multiply and divide. I then created a test class Exercise1_SetupJUnit that takes two numbers as input and tests all four methods. In a real JUnit project, we add the JUnit dependency in pom.xml and use @Test and assertEquals annotations to automate testing. This exercise demonstrates the basic setup and working of unit testing in Java.

Exercise 3: Assertions in JUnit

```
=== JUnit Assertions Demo ===
```

```
Enter first number : 5
Enter second number : 55
```

```
--- Assertion Results ---
```

```
assertEquals : 5 + 55 = 60
assertTrue   : 5 > 55 is false
assertFalse  : 5 < 55 is true
assertNull   : value is null? true
assertNotNull : name is not null? true
```

```
All assertions passed!
```

Explanation :

Assertions in JUnit are used to verify that the actual output of a method matches the expected result. In this exercise, I demonstrated five types of assertions — assertEquals checks if two values are equal, assertTrue checks if a condition is true, assertFalse checks if a condition is false, assertNull checks if a value is null, and assertNotNull checks if a value is not null. I took two numbers as input from the user and applied all five assertions on them to show how each one works. In real JUnit, these assertions automatically pass or fail the test, making it easy to identify bugs in the code.

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and TearDown Methods in Junit

```
=== AAA Pattern - Shopping Cart ===

[SETUP]    Fresh cart created
Enter number of items to add : 5
[ASSERT]   Items in cart = 5
[TEARDOWN] Cart cleared

[SETUP]    Fresh cart created
Enter number of items to remove : 1
[ASSERT]   Items remaining = 4
[TEARDOWN] Cart cleared

All tests passed!
```

Explanation :

The AAA pattern stands for Arrange, Act and Assert — it is the standard way of organizing unit tests in JUnit. In this exercise, I used a Shopping Cart scenario where I tested adding and removing items from the cart. The setUp() method acts as @Before which creates a fresh ShoppingCart object before each test, and tearDown() acts as @After which clears the cart after each test to avoid data interference between tests. In Test 1, I arranged a fresh cart, acted by adding items, and asserted the item count. In Test 2, I pre-filled 5 items, removed some, and verified the remaining count. This pattern ensures every test is independent, clean and easy to debug.

Exercise 1: Mocking and Stubbing

```
=== Mocking and Stubbing Demo ===

Enter city name : Trichy
Weather Result  : No data available

Mock returned data without calling real API!
```

Explanation :

Mocking is a technique used in testing to replace a real external dependency with a fake object that returns predefined values. In this exercise, I created a WeatherApi interface and a MockWeatherApi class that acts as a fake API returning fixed weather data without any real internet connection. The WeatherService uses this mock instead of the real API, and stubbing ensures that when a specific city

like "Chennai" is passed, it always returns "Sunny, 35°C". This allows us to test our service logic independently without depending on external systems.

Exercise 2: Verifying Interactions

```
=== Verifying Interactions Demo ===

Enter user name : Vinusha
Enter item name : Book
Order placed for Vinusha : Book
[MOCK] Message sent to Vinusha : Your order for Book is confirmed!

--- Verification ---
sendMessage called? : true
Called 1 time(s)
Sent to user          : Vinusha
Message               : Your order for Book is confirmed!
```

Explanation :

Verifying interactions in Mockito ensures that a specific method was actually called with the correct arguments during a test. In this exercise, I used an Order and Notification scenario where placing an order should automatically trigger a notification to the user. I created a MockNotificationService that tracks how many times sendMessage() was called, who it was sent to, and what message was sent. After calling placeOrder(), I verified that sendMessage was called exactly once with the correct user and message, confirming that the service interaction worked as expected.

SLF4J logging framework

Exercise 1: Logging Error Messages and Warning Levels

```
Enter user name   : Vinu
Enter amount (Rs.): 50000

Jun 23, 2026 10:35:29 AM Week_1.SLF4J.Logging processTransaction
WARNING: WARNING : Large transaction Rs.50000.0 by Vinu

--- All Log Levels ---
Jun 23, 2026 10:35:29 AM Week_1.SLF4J.Logging main
INFO: INFO      : Application started successfully
Jun 23, 2026 10:35:29 AM Week_1.SLF4J.Logging main
WARNING: WARNING : Low account balance detected
Jun 23, 2026 10:35:29 AM Week_1.SLF4J.Logging main
SEVERE: ERROR   : Database connection failed!
```

Explanation :

SLF4J is a logging framework used in Java to record application events at different severity levels such as INFO, WARNING and ERROR. In this exercise, I built a bank transaction logger where the log level changes based on the transaction amount — a normal transaction logs an INFO message, a large

transaction above Rs.10,000 logs a WARNING, and an invalid amount logs a SEVERE error. I used `java.util.logging` which follows the same concept as SLF4J but requires no extra dependencies. This type of logging helps developers monitor application behaviour, detect issues and debug errors in real-time production environments.